

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
import urllib.request

# -----
# 1. Download & Load Reference Test Image
# -----
url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/chessboard.png"
urllib.request.urlretrieve(url, "test_pattern.png")

img_clean = cv2.imread("test_pattern.png", cv2.IMREAD_GRAYSCALE)
if img_clean is None:
    # Synthetic Fallback Image if download fails
    img_clean = np.ones((400, 400), dtype=np.uint8) * 200
    cv2.rectangle(img_clean, (100, 100), (300, 300), 50, -1)
    cv2.circle(img_clean, (200, 200), 60, 255, -1)

# -----
# 2. Function to Generate Synthetic Noise
# -----
def add_gaussian_noise(image, mean=0, std=25):
    """Adds zero-mean Gaussian noise to the image."""
    gaussian_noise = np.random.normal(mean, std, image.shape).astype(np.float32)
    noisy_img = np.clip(image.astype(np.float32) + gaussian_noise, 0, 255)
    return noisy_img.astype(np.uint8)

def add_salt_and_pepper_noise(image, amount=0.04):
    """Adds salt-and-pepper noise to the image."""
    noisy_img = image.copy()
    # Salt noise
    num_salt = np.ceil(amount * image.size * 0.5)
    coords = [np.random.randint(0, i - 1, int(num_salt)) for i in image.shape]
    noisy_img[tuple(coords)] = 255
    # Pepper noise
    num_pepper = np.ceil(amount * image.size * 0.5)
    coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in image.shape]

```

```

        coords = [np.random.randint(0, 1 - 1, int(num_pepper)) for i in image.shape]
        noisy_img[tuple(coords)] = 0
    return noisy_img

# Generate noisy image variants
img_gauss_noisy = add_gaussian_noise(img_clean, std=30)
img_sp_noisy = add_salt_and_pepper_noise(img_clean, amount=0.05)

# Pre-filtered noisy image (Gaussian Blur applied prior to feature extraction)
img_gauss_filtered = cv2.GaussianBlur(img_gauss_noisy, (5, 5), 1.5)

# -----
# 3. Apply ORB Feature Detector Across All Variants
# -----
orb = cv2.ORB_create(nfeatures=500)

images_list = [img_clean, img_gauss_noisy, img_sp_noisy, img_gauss_filtered]
titles = [
    "1. Clean Reference",
    "2. Gaussian Noise (std=30)",
    "3. Salt & Pepper (5%)",
    "4. Denoised Pre-Filter"
]

results_vis = []
keypoint_counts = []

for img in images_list:
    keypoints, descriptors = orb.detectAndCompute(img, None)
    keypoint_counts.append(len(keypoints))
    # Draw detected keypoints as green circles
    img_kp = cv2.drawKeypoints(
        img, keypoints, None, color=(0, 255, 0),
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )
    results_vis.append(img_kp)

# -----
# 4. Display Visual Results & Quantitative Metrics

```

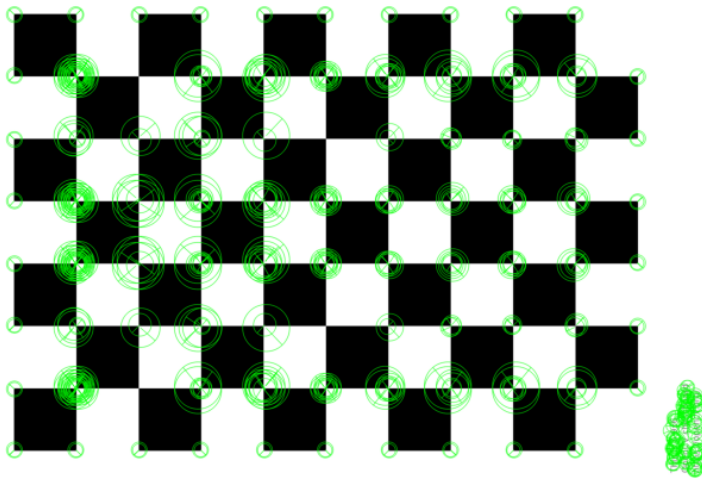
```
# 4. Display visual results & quantitative metrics
# -----
plt.figure(figsize=(16, 10))

for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(results_vis[i])
    plt.title(f"{titles[i]}\nDetected Keypoints: {keypoint_counts[i]}", fontsize=11, fontweight='b')
    plt.axis('off')

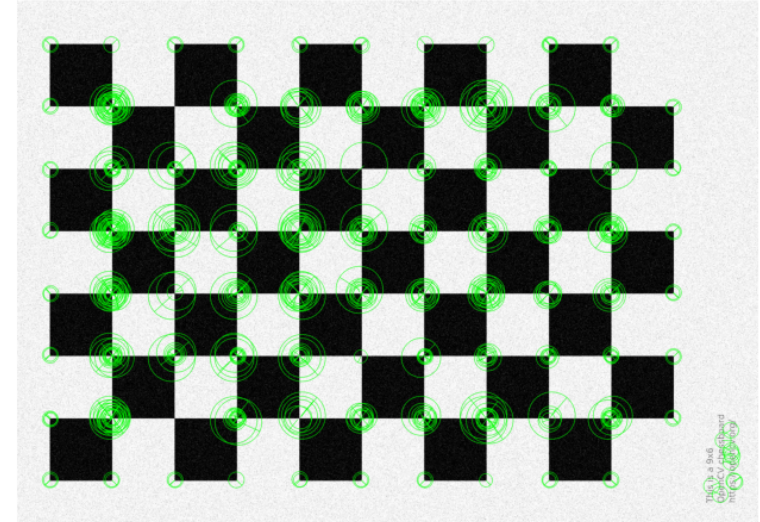
plt.tight_layout()
plt.show()

# Print quantitative comparison summary
print("--- QUANTITATIVE EXPERIMENTAL SUMMARY ---")
for i in range(4):
    print(f"Condition: {titles[i]} | Keypoints Count: {keypoint_counts[i]}")
```

1. Clean Reference
Detected Keypoints: 503



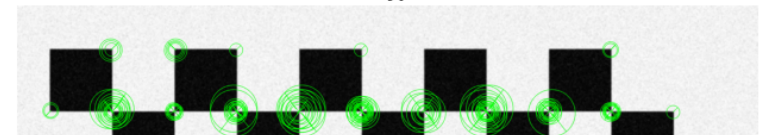
2. Gaussian Noise (std=30)
Detected Keypoints: 500

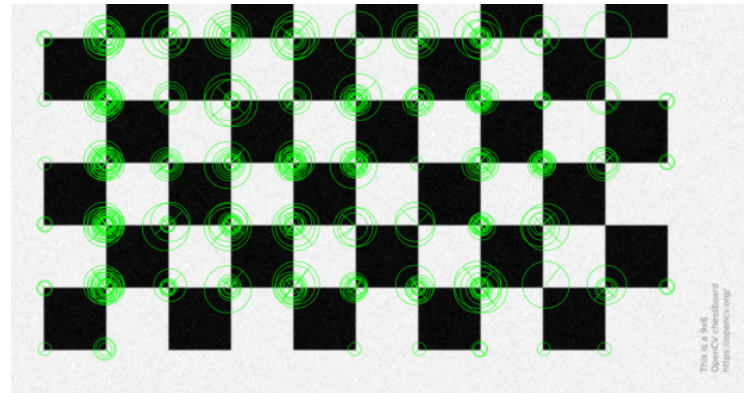
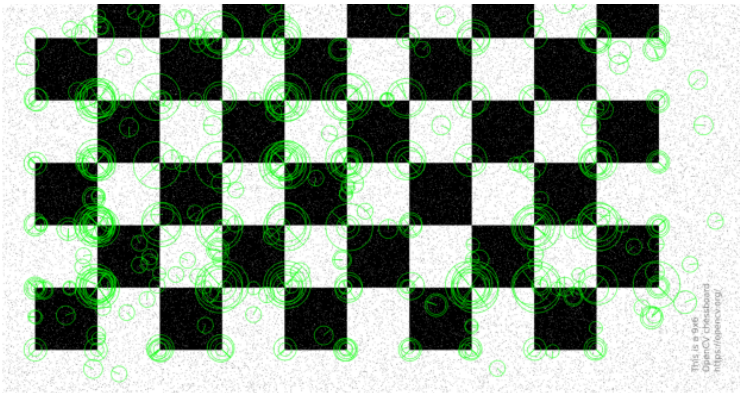


3. Salt & Pepper (5%)
Detected Keypoints: 500



4. Denoised Pre-Filter
Detected Keypoints: 500





--- QUANTITATIVE EXPERIMENTAL SUMMARY ---

Condition: 1. Clean Reference | Keypoints Count: 503

Condition: 2. Gaussian Noise (std=30) | Keypoints Count: 500